■ CINEGO MOVIE TICKET BOOKING SYSTEM

Comprehensive Role Guide: Theatre Manager vs. System Admin

Operational Options, Database Architecture, and Project Viva Presentation Script

1. System Overview & Role Separation (RBAC)

CineGo implements a secure **Role-Based Access Control (RBAC)** architecture separating operational responsibilities. The system distinguishes between internal operational branch duties (**Theatre Manager**) and enterprise platform oversight (**CinemaBook Admin**). This enforces the *Principle of Least Privilege* in database management systems.

Dimension	THEATRE MANAGER	CINEMABOOK ADMIN
Target Scope	Single Theatre Property / Branch (e.g. PVR Grand Mall)	Entire Global Platform (All Theatres, Movies, Cities)
Core Focus	Onboarding, Gate Entry, Occupancy & Daily Concessions	Catalog Governance, Partner Verification, Global Analytics
Web Portal Route	/manager (Manager Command Center)	/admin (Admin Master Console)
CLI / SQL Flow	@database/manager_flow.sql	@database/admin_flow.sql
Key PL/SQL Procedures	REGISTER_THEATRE, VIEW_THEATRE_BOOKINGS, GENERATE_TICKET	ADD_MOVIE, UPDATE_MOVIE, VERIFY_THEATRE, CREATE_SHOW, VIEW_BOOKING_REPORT

2. Theatre Manager: Options & Operations Explained

A Theatre Manager represents the venue owner or operational supervisor of a specific cinema branch. Their capabilities ensure smooth daily theater operations, gate security, and concession management.

Option 1: Register / Onboard Theatre Property

PL/SQL Procedure: REGISTER_THEATRE(...)
What it does: Captures cinema branch legal data including Theatre ID, Name, City Location ID, Physical Address, License Number, GSTIN, Fire Safety NOC, and Duty Manager ID. New registrations start in *'PENDING'* status awaiting central Admin approval.

Option 2: View Theatre Bookings & Revenue

PL/SQL Procedure: VIEW_THEATRE_BOOKINGS(p_theatre_id)
What it does: Executes an explicit cursor joining BOOKINGS, SHOWS, and MOVIES. It outputs ticket sales, customer names, seat numbers, and total revenue earned specifically for that theater.

Option 3: Real-Time Seat Occupancy Audit

PL/SQL Procedure: VIEW_AVAILABLE_SEATS(p_show_id)
What it does: Displays the real-time status of every seat in the auditorium for an active showtime. Shows which seats are AVAILABLE, LOCKED by online customers, or BOOKED.

Option 4: Audit Cafeteria & Food Menu

PL/SQL Procedure: VIEW_FOOD_MENU
What it does: Lists all available cafeteria food items, category (Snacks, Combos, Beverages), pricing, and dietary classification (Veg/Non-Veg) for customer concessions.

Option 5: Turnstile Gate Admission & Ticket Verification

PL/SQL Procedure: GENERATE_TICKET(p_booking_id) & Query on TICKETS
What it does: Scans customer E-tickets or Booking IDs at cinema gate turnstiles. Validates that the ticket is in ACTIVE status, displays seat assignments, and updates status to prevent double-entry.

3. CinemaBook Admin: Options & Governance Explained

The System Administrator possesses platform-wide governance permissions across the entire multi-city catalog, legal verifications, and financial auditing.

Option 1: Add New Movie to Master Catalog

PL/SQL Procedure: ADD_MOVIE(p_movie_id, p_title, p_language, p_genre, p_duration)
What it does: Inserts new cinematic release records into MOVIES table with certification, genre, and duration. Default release status begins as *'RUNNING'* or *'UPCOMING'*.

Option 2: Update Movie Metadata & Lifecycle

PL/SQL Procedure: UPDATE_MOVIE(p_movie_id, p_title, p_status)
What it does: Updates movie titles and shifts screening status from *'UPCOMING'* to *'RUNNING'* to *'COMPLETED'*, directly controlling public visibility in booking searches.

Option 3: Verify & Approve Partner Theatres

PL/SQL Procedure: VERIFY_THEATRE(p_theatre_id)
What it does: Reviews legal license and safety NOC credentials submitted by Theatre Managers. Updates VERIFICATION_STATUS to *'APPROVED'* and activates the property (IS_ACTIVE = 1).

Option 4: Schedule Shows with Automated Seat Matrix Generation

PL/SQL Procedure: CREATE_SHOW(p_show_id, p_movie_id, p_screen_id, p_show_date, p_show_time, p_base_price)
What it does: Creates show screening record AND dynamically iterates through all seats configured in that screen, generating fresh SHOW_SEATS entries in 'AVAILABLE' state in one transaction!

Option 5: Platform-Wide Booking & Revenue Audit Report

PL/SQL Procedure: VIEW_BOOKING_REPORT
What it does: Executes analytical aggregations across all bookings: calculates Gross Merchandise Value (GMV), total discount deductions, platform convenience fee totals, and confirmed ticket counts.

Option 6: Archive / Deactivate Movie (Soft Delete)

PL/SQL Procedure: REMOVE_MOVIE(p_movie_id)
What it does: Soft-deletes movies by toggling status to *'CANCELLED'* rather than destructive DELETE, maintaining referential integrity across prior customer tickets and historical audit tables.

4. How to Explain to Mam (Viva Presentation Script)

Follow this structured 4-step script during your project presentation or external viva examination. It covers your architectural rationale, role distinctions, and technical depth.

Step 1: Introduction & Architecture Overview (Say this first)

*"Respected Ma'am, our project **CineGo** is a next-generation full-stack movie ticket booking system built on an enterprise 3-tier architecture: a high-performance React TypeScript frontend, a Node.js REST API layer, and an Oracle 21c Database engine utilizing PL/SQL procedures, functions, packages, and automated triggers.*

*To enforce data security and business integrity, we implemented **Role-Based Access Control (RBAC)** with three distinct user actors: **Customer**, **Theatre Manager**, and **CinemaBook System Admin**. Today, I would like to demonstrate the operational differences between the Theatre Manager and Admin flows."*

Step 2: Explaining the Theatre Manager Flow (Say this next)

*"Ma'am, the **Theatre Manager** handles the local branch operations for a cinema property like PVR Grand Mall. Their key responsibilities are:*
*1. **Venue Registration:** They submit their cinema details, GSTIN, and safety licenses using the REGISTER_THEATRE stored procedure.*
*2. **Real-Time Occupancy:** Before and during a show, they audit the auditorium seat matrix using VIEW_AVAILABLE_SEATS to monitor real-time occupancy.*
*3. **Revenue Tracking:** They use VIEW_THEATRE_BOOKINGS with an explicit cursor to inspect ticket revenues generated specifically at their branch.*
*4. **Turnstile Gate Verification:** At the entry gate, our system allows the manager to scan the customer's QR ticket. Under the hood, this executes GENERATE_TICKET and queries the TICKETS table to verify the ticket is active and prevent fraudulent double-admissions."*

Step 3: Explaining the CinemaBook Admin Flow (Say this next)

*"In contrast, the **CinemaBook Admin** holds platform-wide governance over all theaters and movies across all cities:*
*1. **Catalog Management:** Using ADD_MOVIE and UPDATE_MOVIE, the Admin maintains release dates, certifications, and movie statuses.*
*2. **Partner Verification:** Admin reviews the safety and license documents submitted by Theatre Managers and approves them via VERIFY_THEATRE.*
*3. **Show & Automated Seat Generation:** When scheduling a screening with CREATE_SHOW, our Oracle procedure automatically loops through all screen seats and populates the SHOW_SEATS table in a single atomic transaction.*
*4. **Platform Auditing & Soft Deletes:** Admin runs VIEW_BOOKING_REPORT for global revenue analytics and archives retired movies via REMOVE_MOVIE with soft-delete semantics to maintain referential integrity with past tickets."*

5. Top 5 Viva Questions & Bulletproof Answers

Q1: Why use PL/SQL stored procedures instead of putting all logic in Node.js / JavaScript?

Answer: *"Ma'am, executing business rules inside Oracle PL/SQL ensures maximum performance and security. It eliminates network round-trips by running logic close to the data, enforces centralized ACID transaction control, and prevents SQL injection because all parameters are strongly typed and bound."*

Q2: How does the system handle concurrent seat selection by two users simultaneously?

Answer: *"Ma'am, we utilize Oracle transaction row-locking. When a user selects a seat, its status shifts from 'AVAILABLE' to 'LOCKED' with a locked timestamp. If another customer attempts to book the same seat simultaneously, our UNIQUE constraint on (SHOW_ID, SEAT_ID) and WHERE STATUS = 'AVAILABLE' clause prevents race conditions and overbooking."*

Q3: What happens in the database when a booking is cancelled?

Answer: *"Ma'am, cancellation triggers our automated trigger TRG_RELEASE_SEAT on the BOOKINGS table. Upon status change to 'CANCELLED', the trigger automatically releases all associated seats back to 'AVAILABLE', marks tickets as 'CANCELLED', and generates an audit record in the REFUNDS table."*

Q4: Why does REMOVE_MOVIE do a soft-delete instead of a SQL DELETE statement?

Answer: *"Ma'am, a hard DELETE would violate Foreign Key referential integrity because existing BOOKINGS and TICKETS reference that MOVIE_ID. Changing STATUS to 'CANCELLED' or 'COMPLETED' preserves historical audit ledgers and financial accounting."*

Q5: How does the Admin show scheduler automatically create seats?

Answer: *"Ma'am, inside CREATE_SHOW, we run an INSERT INTO SHOW_SEATS SELECT ... FROM SEATS WHERE SCREEN_ID = p_screen_id. This copies every physical seat blueprint into the show instance in O(1) database time."*

Project: CineGo Movie Ticket Booking System • **Author:** Ruthra K • **Database:** Oracle Database 21c XE